

Phase 3: Remote Multi-threaded Shell Implementation

COSC 354 : Operating Systems
Final Report

Spring 2026

by

Abi Ali Murtaza Ghadiyali 100066653
Muhammad Awais Afzal Mirza 100066617

Table of Contents:

1. Introduction	2
2. Project Development by Phase	3
2.1 Phase 1 - Local CLI Shell	3
2.2 Phase 2 - Remote CLI Shell.....	3
2.3 Phase 3 - Remote Multitasking CLI Shell.....	3
3. Final File Structure	4
3.1 shell_common.h	4
3.2 shell_client.c.....	4
3.3 shell_server.c.....	4
3.4 Makefile	5
System Design and Implementation	5
4.1 Communication Protocol	5
4.2 Command Parsing and Execution	6
4.3 Session Handling.....	6
4.4 Multithreading Design.....	7
4.5 Shared Server State and Synchronization	7
state_mutex.....	7
log_mutex.....	8
4.6 Capacity Control and Safety	8
5. Build and Run Instructions.....	8
5.1 Build	8
5.2 Start the Server	8
5.3 Start the Client.....	8
5.4 General Usage	9
6. Test Cases and Results	9
7. Output Screenshots and Evidence	10
7.1 Build and Compilation	10
7.2 Server Startup and Multiple Client Connections	11

7.3 Basic Remote Command Execution	11
7.4 File and Directory Operations	12
7.5 Error Handling.....	12
7.6 Pipeline Execution.....	13
7.7 Session Isolation Across Clients.....	14
7.8 Client Connection and Disconnection Logs.....	14
7.9 Server Capacity Limit and Rejection Handling	15
7.10 Exit Command.....	15
7.11 Optional Additional Edge Cases.....	15
8. Discussion and Limitations	16
9. Conclusion.....	16
10. Appendix	17
10.1 Final Source Files	17

1. Introduction

This project presents the design and implementation of a Linux-like shell built progressively in three phases. The work started with a local command-line shell, then extended to a remote client-server shell, and finally became a multithreaded remote shell that can handle multiple clients at the same time.

The final system combines several important operating system concepts:

- process creation using `fork()`
- program execution using `execvp()`
- inter-process communication using `pipe()`
- socket communication using TCP
- multithreading using `pthread`
- synchronization using mutexes

The goal of the project was to create a working shell that supports normal commands, built-in commands, pipelines, remote execution, and concurrent client sessions.

2. Project Development by Phase

2.1 Phase 1 - Local CLI Shell

In Phase 1, the shell worked locally on one machine. The user typed a command, the shell parsed it, and then executed it directly on the operating system.

Main features implemented in Phase 1:

- command parsing
- built-in commands such as `cd`, `clear`, and `exit`
- execution of normal Linux commands
- pipeline support using pipes and multiple processes

This phase introduced the basic shell logic that was reused later in the project.

2.2 Phase 2 - Remote CLI Shell

In Phase 2, the shell was converted into a remote shell using a client-server model.

In this phase:

- the client reads user input
- the client sends the command to the server using sockets
- the server processes the command
- the server sends the result back to the client
- the client prints the result on the screen

This phase introduced networking and remote command execution. It also added session-based directory handling, so the server could keep track of the current working directory for the connected client.

2.3 Phase 3 - Remote Multitasking CLI Shell

In Phase 3, the server was upgraded to support multiple clients at the same time using threads.

In this phase:

- the main server thread accepts incoming connections
- each client is handled in a separate detached worker thread
- each client has its own independent session
- the server can support up to 10 active clients
- if the server is full, the extra client is rejected with a clear message

This phase introduced multithreading, shared server state, synchronization using mutexes, and capacity control.

3. Final File Structure

The final implementation used for submission is stored in the Phase_3 folder.

3.1 `shell_common.h`

This file contains the shared constants and communication structures used by both the client and the server.

It defines:

- port number
- command and output buffer sizes
- maximum number of commands in a pipeline
- maximum number of client connections
- status codes used in communication
- the `CommandResult` structure

Its purpose is to make sure that both the client and the server use the same communication protocol.

3.2 `shell_client.c`

This file implements the client side of the project.

Its responsibilities are:

- connect to the server
- read commands from the user
- send commands to the server
- receive results from the server
- display output to the user
- handle special results such as `clear`, `exit`, and `server full`

3.3 `shell_server.c`

This file implements the main logic of the shell system.

Its responsibilities are:

- create and manage the server socket
- accept client connections
- create one worker thread per client

- maintain a separate session for each client
- process commands
- execute built-in commands, single commands, and pipeline commands
- send results back to the correct client

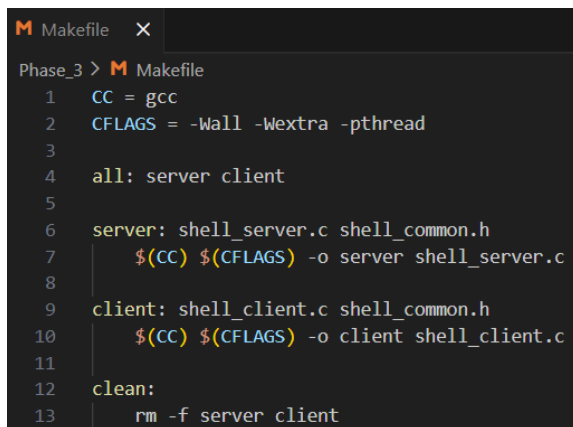
3.4 Makefile

This file automates compilation of the project.

It provides:

- a build target for the server
- a build target for the client
- a clean target to remove binaries

This makes the build process simple and consistent.



```

M Makefile x
Phase_3 > M Makefile
1  CC = gcc
2  CFLAGS = -Wall -Wextra -pthread
3
4  all: server client
5
6  server: shell_server.c shell_common.h
7      $(CC) $(CFLAGS) -o server shell_server.c
8
9  client: shell_client.c shell_common.h
10     $(CC) $(CFLAGS) -o client shell_client.c
11
12  clean:
13      rm -f server client
  
```

System Design and Implementation

4.1 Communication Protocol

The client and server communicate using TCP sockets.

The communication model is simple:

1. the client sends one fixed-size command buffer to the server
2. the server processes the command
3. the server sends back one `CommandResult` structure
4. the client reads the result and displays it

To make communication reliable, both sides use helper functions:

- `send_all()` sends all bytes even if `send()` writes only part of them
- `recv_all()` receives all bytes even if `recv()` returns partial data

In Phase 3, a startup handshake was added:

- a normal client receives `STATUS_OK`
- if the server is full, the client receives `STATUS_SERVER_FULL` and a short rejection message

This makes the overflow case clear and easy to test.

4.2 Command Parsing and Execution

The shell uses the following main steps to handle commands:

- `parse_line()` splits the input line by the pipe symbol `|`
- `parse_command()` splits each command into arguments
- `process_command()` decides whether the input is:
 - a blank command
 - a built-in command
 - a single command
 - a piped command

Built-in commands are handled directly inside the server. These include:

- `cd`
- `clear`
- `exit`

Single external commands are executed using:

- `fork()`
- `execvp()`
- `waitpid()`

Pipelines are executed using:

- multiple child processes
- multiple pipes
- `dup2()` to redirect input and output

All command output is captured by the server and returned to the client using the `CommandResult` structure.

4.3 Session Handling

Each client has its own session represented by a `Session` structure.

The session stores:

- the client socket
- the client ID
- the current session directory
- the active status of the session

This design is important because each client must have its own working directory. If one client changes directory using `cd`, it should not affect the directory of another client.

4.4 Multithreading Design

The Phase 3 server uses a thread-per-client model.

The design works as follows:

- the main thread creates the listening socket
- the main thread keeps accepting new client connections
- for each accepted client, the server creates a detached worker thread
- the worker thread handles all communication and command execution for that client

This allows the server to support several clients at the same time.

4.5 Shared Server State and Synchronization

The server also maintains a shared `ServerState` structure.

This structure stores:

- the number of active clients
- the next available client ID
- a mutex for shared state
- a mutex for logging

Two mutexes are used:

`state_mutex`

This mutex protects shared variables such as:

- `active_clients`
- `next_client_id`

This prevents race conditions when multiple threads try to register or unregister clients at the same time.

`log_mutex`

This mutex protects server log output.

Without this mutex, messages from multiple threads could mix together on the terminal and become hard to read.

4.6 Capacity Control and Safety

The final system supports a maximum of 10 active clients at the same time.

If the server is full:

- the new client is rejected
- the server sends a short rejection message
- the client exits cleanly

The server also ignores SIGPIPE so that a broken client connection does not crash the whole server.

5. Build and Run Instructions

5.1 Build

From the Phase_3 directory, run:

```
make clean  
make
```

5.2 Start the Server

Open one terminal and run:

```
./server
```

5.3 Start the Client

Open another terminal and run:

```
./client
```

5.4 General Usage

After connecting, the client shows a shell prompt:

```
shell>
```

The user can then enter:

- normal commands such as `pwd`, `ls`, `mkdir`, and `rm -r`
- built-in commands such as `cd`, `clear`, and `exit`
- piped commands such as `ls | wc -l`

6. Test Cases and Results

The following test cases were used to verify the project.

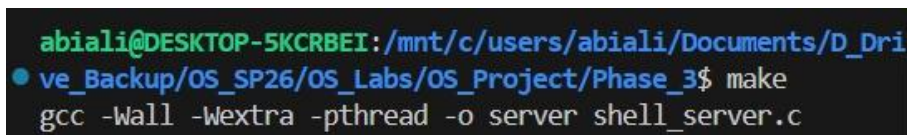
Test ID	Feature Tested	Example Command(s)	Expected Result	Observed Result
T1	Build	<code>make clean && make</code>	Client and server compile successfully	Passed
T2	Basic connection	<code>./server</code> , then <code>./client</code>	Client connects to server successfully	Passed
T3	Basic command execution	<code>pwd</code> , <code>ls</code>	Correct directory and file list shown	Passed
T4	Directory creation	<code>mkdir testing1</code>	New directory is created	Passed
T5	Directory navigation	<code>cd testing1</code> , <code>pwd</code>	Client session moves into the directory	Passed
T6	File/directory removal	<code>rm -r testing1</code>	Directory is removed successfully	Passed
T7	Error handling - missing <code>cd</code> arg	<code>cd</code>	Error message shown	Passed
T8	Error handling - too many <code>cd</code> args	<code>cd a b</code>	Error message shown	Passed
T9	Error handling - invalid path	<code>cd fake_dir</code>	Error message shown	Passed
T10	Error handling - invalid command	<code>fakecommand</code>	Error message shown	Passed

Test ID	Feature Tested	Example Command(s)	Expected Result	Observed Result
T11	One-pipe command	<code>ls wc -l</code>	Final result returned correctly	Passed
T12	Multi-pipe command	<code>ls grep shell wc -l</code>	Final result returned correctly	Passed
T13	Maximum pipeline case	<code>echo one two three four five wc -w cat tail -n 1 head -n 1</code>	Correct value returned using maximum supported pipeline length	Passed
T14	Exit command	<code>exit</code>	Session closes cleanly	Passed
T15	Multiple clients at once	open several <code>./client</code> sessions	Server accepts concurrent clients	Passed
T16	Session isolation	one client uses <code>cd</code> , another runs <code>pwd</code>	Each client keeps its own directory state	Passed
T17	Capacity control	connect more than 10 clients	Extra client is rejected with a clear message	Passed
T18	Port reuse protection	start a second server on same port	Bind failure message appears	Passed

7. Output Screenshots and Evidence

The screenshots below are grouped by feature so that each one clearly shows a different part of the system working.

7.1 Build and Compilation



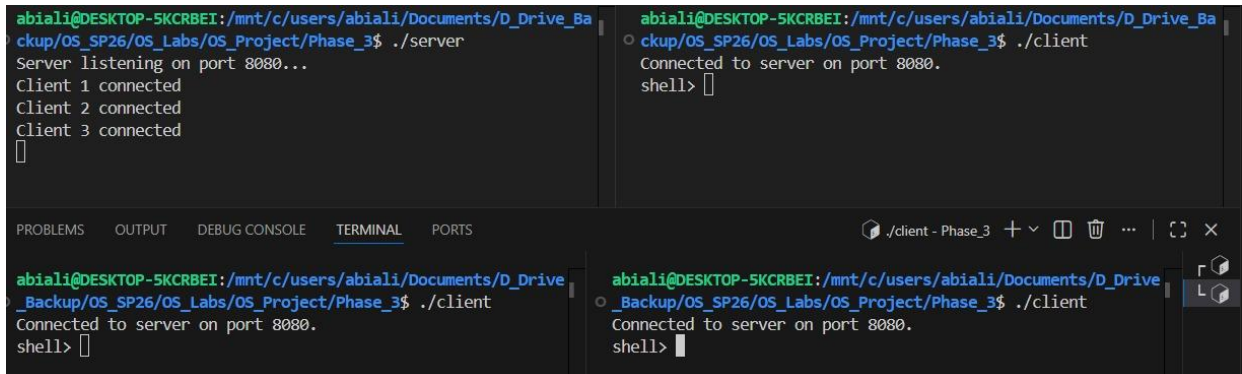
```

abiali@DESKTOP-5KCRBEI:/mnt/c/users/abiali/Documents/D_Dri
ve_Backup/OS_SP26/OS_Labs/OS_Project/Phase_3$ make
gcc -Wall -Wextra -pthread -o server shell_server.c

```

What this shows: the project compiles successfully using the provided Makefile.

7.2 Server Startup and Multiple Client Connections



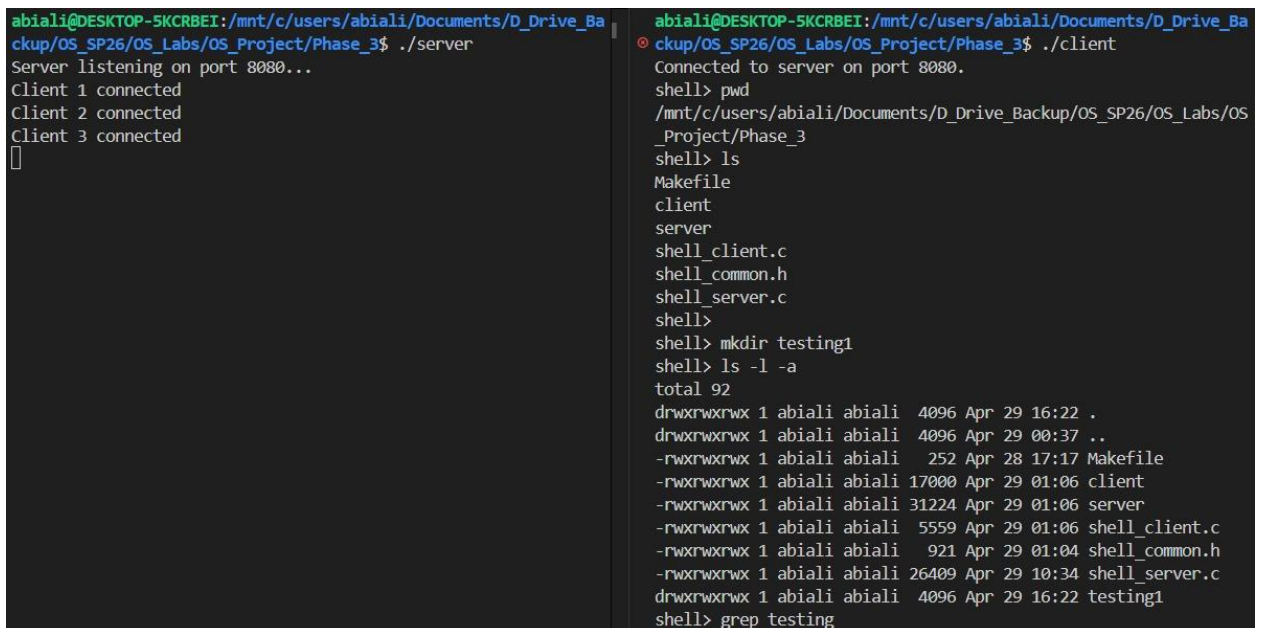
The screenshot shows a terminal window with two panes. The left pane shows the server running and accepting three client connections. The right pane shows a client connected to the server and entering a shell prompt. Below the panes, there are tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The TERMINAL tab is active, showing the same server and client interactions as the panes above.

```
abiali@DESKTOP-5KCRBEI:/mnt/c/users/abiali/Documents/D_Drive_Backup/OS_SP26/OS_Labs/OS_Project/Phase_3$ ./server
Server listening on port 8080...
Client 1 connected
Client 2 connected
Client 3 connected
[]

abiali@DESKTOP-5KCRBEI:/mnt/c/users/abiali/Documents/D_Drive_Backup/OS_SP26/OS_Labs/OS_Project/Phase_3$ ./client
Connected to server on port 8080.
shell> []
```

What this shows: the server starts correctly, listens on port 8080, and accepts multiple clients.

7.3 Basic Remote Command Execution



The screenshot shows a terminal window with two panes. The left pane shows the server running and accepting three client connections. The right pane shows a client connected to the server and executing several shell commands: pwd, ls, mkdir testing1, ls -l -a, and grep testing. The output of these commands is shown in the right pane.

```
abiali@DESKTOP-5KCRBEI:/mnt/c/users/abiali/Documents/D_Drive_Backup/OS_SP26/OS_Labs/OS_Project/Phase_3$ ./server
Server listening on port 8080...
Client 1 connected
Client 2 connected
Client 3 connected
[]

abiali@DESKTOP-5KCRBEI:/mnt/c/users/abiali/Documents/D_Drive_Backup/OS_SP26/OS_Labs/OS_Project/Phase_3$ ./client
Connected to server on port 8080.
shell> pwd
/mnt/c/users/abiali/Documents/D_Drive_Backup/OS_SP26/OS_Labs/OS_Project/Phase_3
shell> ls
Makefile
client
server
shell_client.c
shell_common.h
shell_server.c
shell>
shell> mkdir testing1
shell> ls -l -a
total 92
drwxrwxrwx 1 abiali abiali 4096 Apr 29 16:22 .
drwxrwxrwx 1 abiali abiali 4096 Apr 29 00:37 ..
-rwxrwxrwx 1 abiali abiali 252 Apr 28 17:17 Makefile
-rwxrwxrwx 1 abiali abiali 17000 Apr 29 01:06 client
-rwxrwxrwx 1 abiali abiali 31224 Apr 29 01:06 server
-rwxrwxrwx 1 abiali abiali 5559 Apr 29 01:06 shell_client.c
-rwxrwxrwx 1 abiali abiali 921 Apr 29 01:04 shell_common.h
-rwxrwxrwx 1 abiali abiali 26409 Apr 29 10:34 shell_server.c
drwxrwxrwx 1 abiali abiali 4096 Apr 29 16:22 testing1
shell> grep testing
```

What this shows: basic shell commands execute correctly through the remote server.

7.4 File and Directory Operations

```
shell> ls
Makefile
client
server
shell_client.c
shell_common.h
shell_server.c
testing1
shell> ls
Makefile
client
server
shell_client.c
shell_common.h
shell_server.c
shell>
shell> rm -r testing1
shell>
```

What this shows: directory creation and deletion work correctly through the shell.

7.5 Error Handling

```
abiali@DESKTOP-5KCRBEI:/mnt/c/users/abiali/Documents/D_Drive_Backup/OS_SP26/OS_Labs/OS_Project/Phase_3$ ./client
Connected to server on port 8080.
shell> cd
shell: cd: no argument provided
shell> cd a b
shell: cd: too many arguments
shell> cd fake_dir
shell: cd: No such file or directory
shell> fakecommand
shell: fakecommand: No such file or directory
shell> exit
Session closed.
```

What this shows: the shell handles invalid input and incorrect commands safely and clearly.

7.6 Pipeline Execution

```
abiali@DESKTOP-5KCRBEI:/mnt/c/users/abiali/Documents/D_Drive_Backup/OS_SP26/OS_Labs/OS_Project/Phase_3$ ./client
Connected to server on port 8080.
shell> ls | wc -l
6
shell> ls | grep shell | wc -l
3
shell> ls
Makefile
client
server
shell_client.c
shell_common.h
shell_server.c
shell>
```

What this shows: the shell supports piped command execution using multiple commands.

```
abiali@DESKTOP-5KCRBEI:/mnt/c/users/abiali/Documents/D_Drive_Backup/OS_SP26/OS_Labs/OS_Project/Phase_3$ ./server
Server listening on port 8080...
Client 1 connected
Client 1 disconnected
[]

abiali@DESKTOP-5KCRBEI:/mnt/c/users/abiali/Documents/D_Drive_Backup/OS_SP26/OS_Labs/OS_Project/Phase_3$ ./client
Connected to server on port 8080.
shell> echo one two three four five | wc -w | cat | tail -n 1 | head -n 1
5
shell> exit
Session closed.
```

What this shows: the shell can execute the longest supported pipeline correctly.

7.7 Session Isolation Across Clients

```
abiali@DESKTOP-5KCRBEI:/mnt/c/users/abiali/Documents/D_Drive_Backup/OS_SP26/OS_Labs/OS_Project/Phase_3$ ./server
Server listening on port 8080...
Client 1 connected
Client 2 connected
Client 3 connected
Client 4 connected
Client 4 disconnected
Client 5 connected
Client 5 disconnected
Client 6 connected
[]

shell> pwd
/mnt/c/users/abiali/Documents/D_Drive_Backup/OS_SP26/OS_Labs/OS_Project/Phase_3
shell> cd ..
/mnt/c/users/abiali/Documents/D_Drive_Backup/OS_SP26/OS_Labs/OS_Project
shell> ls
# Spring26Project-Assignments-adjus.md
OS Phase 1 Report + Code.docx
OS Phase 1 Report + Code.pdf
Phase 2 plan.docx
Phase 2 plan.md
Phase_2
Phase_3
Spring26Project-Assignments-adjusted-april-v2.pdf
shell
shell.c
shell_client
shell_client.c
shell_server
shell_server.c
shell> []
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
abiali@DESKTOP-5KCRBEI:/mnt/c/users/abiali/Documents/D_Drive_Backup/OS_SP26/OS_Labs/OS_Project/Phase_3$ ./client
Connected to server on port 8080.
shell> pwd
/mnt/c/users/abiali/Documents/D_Drive_Backup/OS_SP26/OS_Labs/OS_Project/Phase_3
shell> ls
Makefile
client
server
shell_client.c
shell_common.h
shell_server.c
testing1
shell> []
```

```
abiali@DESKTOP-5KCRBEI:/mnt/c/users/abiali/Documents/D_Drive_Backup/OS_SP26/OS_Labs/OS_Project/Phase_3$ ./client
Connected to server on port 8080.
shell> pwd
/mnt/c/users/abiali/Documents/D_Drive_Backup/OS_SP26/OS_Labs/OS_Project/Phase_3
shell> []
```

What this shows: each client session has its own current working directory and does not affect the others.

7.8 Client Connection and Disconnection Logs

```
abiali@DESKTOP-5KCRBEI:/mnt/c/users/abiali/Documents/D_Drive_Backup/OS_SP26/OS_Labs/OS_Project/Phase_3$ ./server
Server listening on port 8080...
Client 1 connected
Client 2 connected
Client 3 connected
Client 4 connected
Client 4 disconnected
Client 5 connected
Client 5 disconnected
[]
```

What this shows: the multithreaded server properly tracks client lifecycle events.

7.9 Server Capacity Limit and Rejection Handling

```
abiali@DESKTOP-5KCRBEI:/mnt/c/users/abiali/Documents/D_Drive_Backup/OS_SP26/OS_Labs/OS_Project/Phase_3$ ./server
Server listening on port 8080...
Client 1 connected
Client 2 connected
Client 3 connected
Client 4 connected
Client 5 connected
Client 6 connected
Client 7 connected
Client 8 connected
Client 9 connected
Client 10 connected
Server full, rejected client
[]
```

What this shows: the server correctly enforces the MAX_CLIENTS limit.

```
abiali@DESKTOP-5KCRBEI:/mnt/c/users/abiali/Documents/D_Drive_Backup/OS_SP26/OS_Labs/OS_Project/Phase_3$ ./client
server full, try again later
```

What this shows: the extra client receives a clear rejection message instead of failing silently.

7.10 Exit Command

```
abiali@DESKTOP-5KCRBEI:/mnt/c/users/abiali/Documents/D_Drive_Backup/OS_SP26/OS_Labs/OS_Project/Phase_3$ ./client
Connected to server on port 8080.
shell> exit
Session closed.
```

What this shows: the session terminates cleanly when the user exits.

7.11 Optional Additional Edge Cases

```
abiali@DESKTOP-5KCRBEI:/mnt/c/users/abiali/Documents/D_Drive_Backup/OS_SP26/OS_Labs/OS_Project/Phase_3$ ./server
Server listening on port 8080...
[]
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
abiali@DESKTOP-5KCRBEI:/mnt/c/users/abiali/Documents/D_Drive_Backup/OS_SP26/OS_Labs/OS_Project/Phase_3$ ./server
server: bind failed: Address already in use
```

What this shows: the same port cannot be bound by two servers at the same time.

```
shell> seq 8 | tail -n 6 | head -n 4 | tail -n 1  
6  
shell> exit  
Session closed.
```

What this shows: another successful multi-pipe test case.

8. Discussion and Limitations

Although the project works correctly for the required scope, there are still some limitations.

- The parser is simple and does not fully support advanced shell syntax such as quoted strings.
- The command buffer and output buffer are fixed in size.
- The server supports a fixed maximum of 10 active clients.
- The project does not implement the optional time-based scheduler idea mentioned generally in the assignment document.

These choices were made intentionally to keep the implementation simple, stable, and aligned with the main phase requirements.

9. Conclusion

This project successfully built a Linux-like shell in three progressive phases.

- Phase 1 introduced local shell execution, built-in commands, and pipelines.
- Phase 2 introduced remote execution through sockets using a client-server model.
- Phase 3 introduced multithreading, synchronization, session isolation, and support for multiple concurrent clients.

The final system is a fully functional multithreaded remote shell that demonstrates important operating system concepts in one integrated project. It supports command execution, built-in commands, pipelines, networking, concurrent clients, and safe communication between client and server.

10. Appendix

10.1 Final Source Files

The final implementation is stored in the submitted `OS_Project.zip` folder and includes:

- `shell_common.h`
- `shell_client.c`
- `shell_server.c`
- `Makefile`